

2026년 7월 월간 공유회

모바일 매니플레이터 로봇 프로젝트 비전 기반 실내 위치 추정

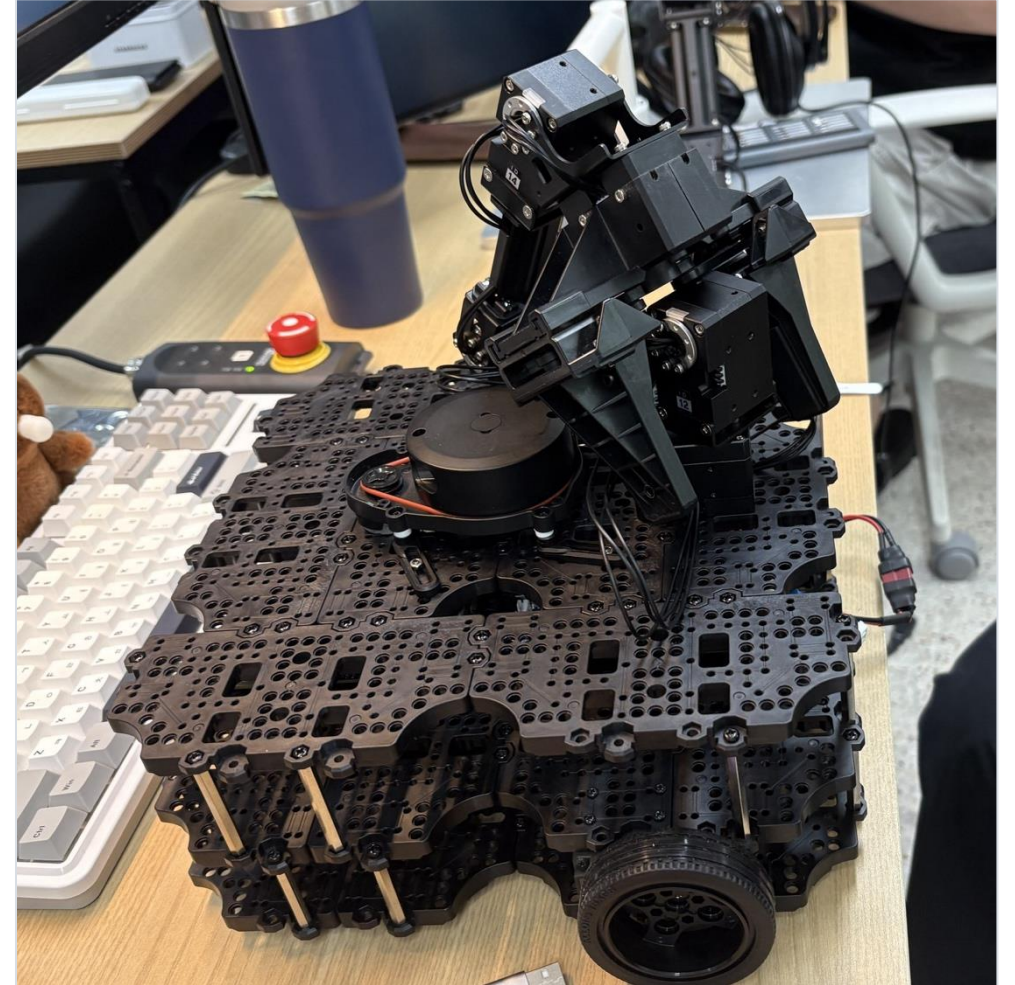
TEAM: 김김정변 | 변지민

목차

- ✓ 이번 달 한 일
- ✓ 매니퓰레이터 & 모바일 플랫폼
- ✓ ArUco 마커 인식
- ✓ 마커 기반 절대 위치 계산
- ✓ 다음 계획

이번 달 한 일

- ✓ 매니퓰레이터(OpenManipulator-X) 조립하고 플랫폼 위에 장착 완료
- ✓ 플랫폼은 직접 프린팅해서 만들려던 걸 터틀봇 기반으로 변경
- ✓ 제어도 맥미니에서 젯슨 오린 나노로 변경
- ✓ ArUco 마커로 카메라 보정부터 위치 계산 테스트 완료
- ✓ 조립은 끝났고, 리얼센스와 아르고마커용 웹캠 파츠만 제작하면 하드웨어 끝



조립을 마친 로봇

ArUco 마커 인식

- ✓ 웹캠(APC900)으로 사진 23장을 찍어 렌즈 왜곡을 보정하고 카메라 값을 뽑음
- ✓ 0·1·2번 마커가 잘 잡히는지 확인하고, solvePnP로 카메라와 마커 사이의 거리·방향까지 구하기
테스트 완료

```
import cv2
import numpy as np

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# 체크보드 코너 찾기
ret, corners = cv2.findChessboardCorners(gray, CHECKBOARD_SIZE, None)

# 코너를 성공적으로 찾았다면
if ret:
    objpoints.append(objp)

    # 체크보드 단위보다 더 정밀하게(사분할로 단위) 코너 위치 조정
    corners2 = cv2.cornerSubPix(gray, corners, (11, 11), (-1, -1),
                                criteria=(cv2.TERM_CRITERIA_EPS + cv2.TERM_

    imgpoints.append(corners2)

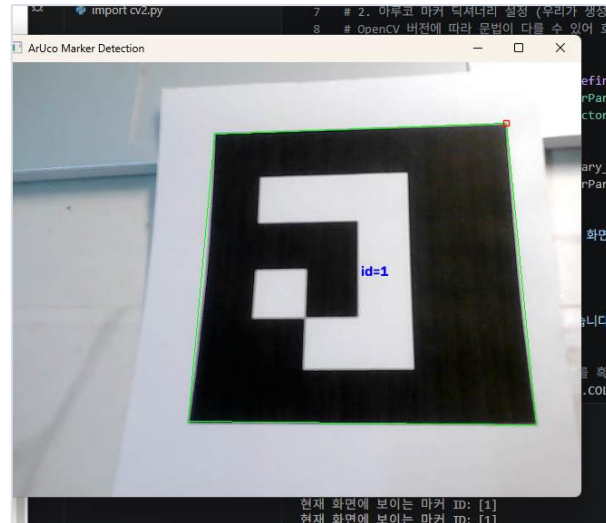
cv2.destroyAllWindows()

# 이미지 분석을 바탕으로 캘리브레이션 수행
print("\n캘리브레이션 연산 중... (수 초 정도 걸릴 수 있음)")
ret, mtx, dist, rvecs, tvecs = cv2.calibrateCamera(objpoints, imgpoints, gray.size(), None, None)

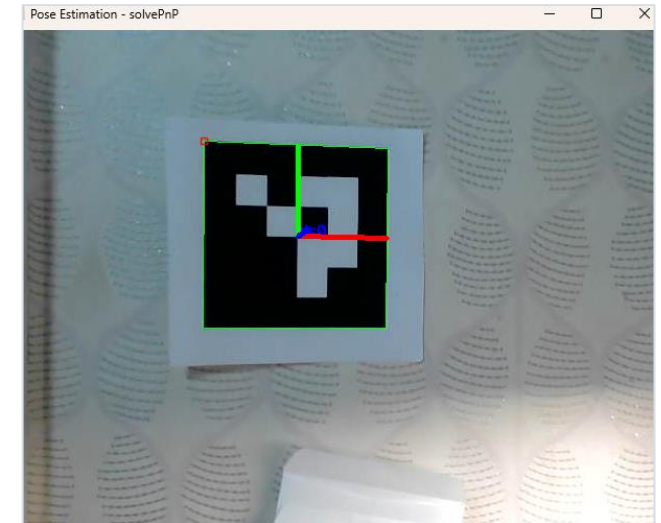
# 결과 출력
print("\n===== 캘리브레이션 완료! =====")
print("[1] 카메라 행렬 (Camera Matrix): mtx")
print("[2] 왜곡 계수 (Distortion Coefficients): dist")

# 나중에 이후로 다시 코드에서 쉽게 불러다 쓸 수 있도록 넘파이 파일로 저장
np.savez('calib_data.npz', mtx=mtx, dist=dist)
print("\n결과가 'calib_data.npz' 파일로 안전하게 저장되었습니다!")
```

카메라 보정 결과



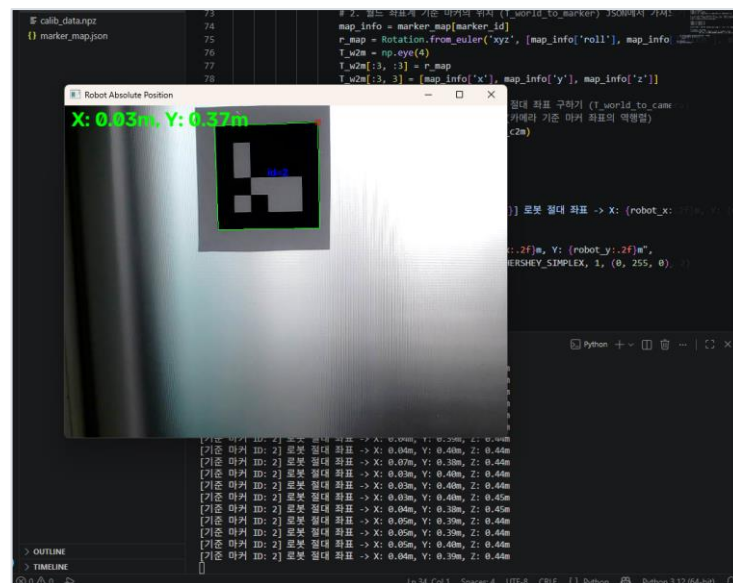
마커 인식 확인



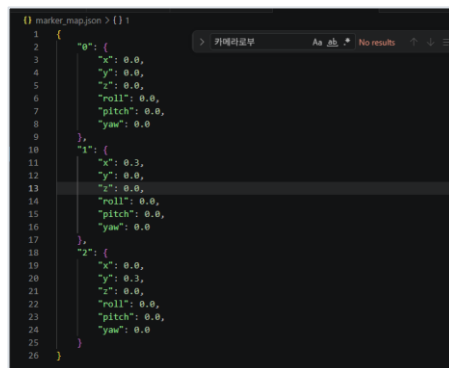
마커까지의 위치 추정

마커 기반 절대 위치 계산

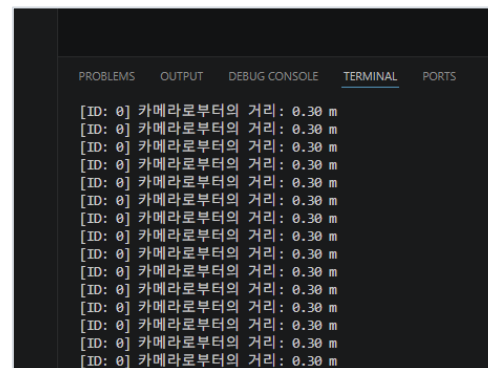
- ✓ 미리 정해둔 마커 위치와 방금 구한 상대 위치를 합쳐서 로봇이 실제로 어디 있는지 계산
- ✓ 원점 마커가 화면에 안 보여도, 다른 마커 하나만 잡히면 위치를 계속 알 수 있음
- ✓ 지금 마커 좌표는 임시로 넣은 값이라, 실제 주행 공간에 맞춰 다시 잡을 계획



실시간 위치 출력 (X 0.03m, Y 0.37m)



마커 좌표 파일



마커까지 거리 로그

다음 계획

- ✓ 센서 붙일 파츠 설계를 끝내고 플랫폼에 달기
- ✓ 젯슨 오린 나노로 플랫폼과 팔을 함께 움직여 보기
- ✓ 실제 주행 공간에서 마커 좌표를 다시 잡고 위치 정확도 확인하기